Qiskit 101: Installation Guide

We require that your work be submitted in a Jupyter notebook format along with a short 5-min video describing your project for ease of grading your submissions. The notebook format also allows you to input markdown cells describing your thought process in solving the challenge and/or hackathon prompts.

We recommend using Google Colab for easily running Jupyter notebooks and sharing the notebooks with your team members. Colab is a free, cloud-based Jupyter Notebook environment that allows for direct execution of python code in your web browser removing the need to locally set up Python on your machine. You can access Colab by logging in with your CU email.

Package installations

Here is a list of the main packages that should be installed for solving the challenges and hackathon prompts:

- pip install **qiskit[visualization]**
- pip install **qiskit-aer**
- pip install **pyscf**
- pip install **pylatexenc**
- pip install **qiskit-nature**
- pip install **qiskit-algorithm**
- pip install **qiskit_ibm_runtime**
- pip install **qc_grader**

The main Qiskit SDK has supporting modules like **qiskit_aer** for noiseless simulations and visualization tools. For running experiments on real hardware, you will use **qiskit_ibm_runtime**. For the quantum chemistry prompt, you will specifically need **qiskit_nature** and **pyscf** to run the provided notebooks and solve the hackathon prompts.

Run Jupyter notebooks on Google Colab

1. Go to Google Colab and log in with your CU credentials.
2. Upload an example notebook provided to you for reference or start your own notebook.
3. Ensure that you install the necessary packages every time you run a notebook for the first time or restart the kernel, for example:
   %pip install qiskit qiskit_aer qiskit_ibm_runtime
   %pip install qiskit[visualization]
   %pip install qiskit_nature pyscf qiskit_algorithms

Step-by-step walkthrough to run code on real hardware

1. Check your inbox for an email with the subject: "**Action required: You are invited to join an account in IBM Cloud**" and click the "**Join now**" or accept link inside the email.
2. Log in or create an account using the email address you provided in the TEAM FORM. DO NOT enter credit card information to create the account.
3. Go directly to the IBM Quantum Cloud Home dashboard and look for the API management

section.

4. Click the **Create an API Key** button and give your key a recognizable name and then click **Create**.
5. Copy the 44-character API key immediately using the clipboard icon and save it somewhere!
6. Open the **Secrets** panel (key icon) in the left sidebar of Google Colab and create a new secret named '**IBM_QUANTUM_API_KEY**'.
7. Paste your API key into the value field and enable **Notebook access** for this secret.
8. Then, run the authentication cell below:

```
from google.colab import userdata
from qiskit_ibm_runtime import QiskitRuntimeService
import os

# Retrieve API key from Colab Secrets
try:
    api_key = userdata.get("IBM_QUANTUM_API_KEY")
except Exception as e:
    raise RuntimeError(
        "Please add IBM_QUANTUM_API_KEY to Colab Secrets "
        "and enable notebook access."
    ) from e

if not api_key:
    raise ValueError("IBM_QUANTUM_API_KEY is empty.")

# Configure IBM Quantum account
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token=api_key,
    overwrite=True,
    set_as_default=True
)

# Also make credentials available to the grader
os.environ["QC_API_KEY"] = api_key

# Initialize and verify service
service = QiskitRuntimeService()

del api_key

print("IBM Quantum account configured successfully!")
```

Please note that hardware time is limited to 10 minutes under the invited accounts. Keep that in mind when planning the circuits you want to run on the hardware.

Track 1: Quantum Chemistry

IBM Sponsored Hackathon Prompt from Basque Quantum

Background and Motivation

The energy of a molecule as a function of the positions of its nuclei - its potential energy surface (PES) - is one of the most useful objects in chemistry. Its minimum tells us the equilibrium geometry; its curvature encodes vibrational frequencies; the way it flattens as atoms separate describe how bonds break. Computing it accurately means solving the electronic structure problem repeatedly, and the classical cost of doing so grows steeply with system size.

Quantum computers offer a natural route: a molecule's electronic state can be encoded into qubits and its energy estimated variationally. IBM Quantum's 2017 demonstration of a hardware-efficient variational quantum eigensolver (VQE) for small molecules showed that real superconducting hardware can estimate molecular ground-state energies. A decade into the era of cloud-accessible quantum computers, this challenge invites you to go beyond a single energy and map out the physics of a bond.

The scientific questions

1. How does the ground state energy of a molecule change as we stretch, compress, and break its bonds?
2. How far can a quantum-computer potential energy surface be trusted as the system grows?

The problem

For a molecule with fixed nuclei, the electronic structure problem is to find the lowest eigenvalue E_0 of the electronic Hamiltonian $\hat{H}$, i.e. the ground-state energy. VQE approaches this through the variational principle: for any parametrized trial state $|\psi(\vec{\theta})\rangle$,

$$E(\vec{\theta}) = \frac{\langle\psi(\vec{\theta})|\hat{H}|\psi(\vec{\theta})}{\langle\psi(\vec{\theta})|\psi(\vec{\theta})\rangle} \geq E_0$$

so minimizing $E(\vec{\theta})$ over the parameters approximates the ground state from above. A potential energy surface is obtained by repeating this minimization at a series of nuclear geometries and plotting E_0 against a geometric coordinate (e.g. a bond length).

Because the challenge is about which molecule and which geometry, each level is specified by a target molecule rather than by an explicit Hamiltonian - you construct the Hamiltonian from the molecule, basis set, and mapping of your choice. The companion tutorial notebook does this end to end for H_2 at its equilibrium bond length.
Additional Jupyter notebooks,

- **01_sample_then_diagonalize.ipynb**
- **02_molecules_as_bitstrings.ipynb**
- **03_the_full_sqd_pipeline.ipynb**

are provided for reference on this track and for further background on sample-based quantum diagonalization (SQD), which is the algorithm behind some of the largest quantum chemistry experiments that ran on real quantum hardware.

Getting started - Beginner

Your target

Compute and plot the ground-state potential energy surface of the HeH^+ molecular ion: its total ground-state energy E_0 as a function of the he He-H internuclear distance, across a range that captures both the equilibrium minimum and the onset of dissociation. Identify the equilibrium bond length from your curve.

Suggested milestones

These are optional pointers, not requirements - how you reach the target is up to you.

- Adapt the tutorial notebook **QChemistry_Beginner_Challenge_Tutorial.ipynb** workflow to HeH^+ (mind the +1 charge and the electron count).
- Sweep the internuclear distance and solve for the ground state at each point.
- Locate the minimum of the curve and read off the equilibrium bond length.
- Comment on the shape near equilibrium and as the atoms are pulled apart.

Next Steps - Intermediate and Advanced

The two levels below are open-ended: the target defines what to achieve, and the milestones merely suggest how.

Intermediate - Do more with less

Your target

Compute the ground-state energy of a larger molecule - LiH or BeH_2 - to within chemical accuracy ($\approx 1.6 \times 10^{-3}$ Ha), while reducing the quantum resources required compared to a naive full-space UCCSD calculation. Report both the energy you reach and the resources (qubits, circuit depth, or measurements) you saved.

Suggested milestones

- Explore levers such as alternative fermion-to-qubit mappings, symmetry-based qubit tapering, freezing core orbitals, or grouping commuting measurements.
- Quantify the trade-off: report accuracy achieved against qubits, circuit depth, or shot budget.
- Decide which combination of techniques buys the most under a fixed budget, and explain why.

Advanced - Beyond a single ansantz

Your target

Estimate the ground-state energy of the nitrogen molecule (N_2) near its equilibrium bond length - a strongly correlated system where a naive UCCSD run struggles - using a sampling/beyond-ansatz approach that lets classical post-processing do the heavy lifting. Show that your result improves on what a plain variational ansatz achieves for the same system.

Suggested milestones

- Choose your strategy (e.g. sample-based quantum diagonalization) and justify why it suits a strongly correlated molecule like N_2.
- Compare your result against a plain UCCSD run on the same system to show the improvement.
- Validate correctness where exact diagonalization is infeasible, e.g. against a classical CI / CCSD reference.

Useful Resources

Qiskit Nature (quantum chemistry)

- Qiskit Nature documentation

Entry point for building electronic-structure problems, drivers, mappers, and chemistry ansätze.

- Ground-state solvers tutorial

Step-by-step tutorial on finding molecular ground states with VQE and classical solvers — the closest reference to this notebook.

- Electronic structure problem tutorial

Explains drivers, basis sets, and how the second-quantized Hamiltonian is constructed.

- Qubit mappers tutorial

Compares Jordan-Wigner, parity, and Bravyi-Kitaev mappings, including qubit tapering via symmetries — directly relevant to the intermediate challenge.

- UCCSD ansatz documentation

Reference page for the UCCSD variational form used in this notebook.

Variational algorithms and optimizers

- VQE documentation Reference page for the Variational Quantum Eigensolver class.
- Qiskit Algorithms optimizers

Reference page for classical optimizers such as 'SLSQP', 'COBYLA', 'SPSA', and others to experiment with.

- AdaptVQE how-to

Builds an ansatz adaptively, a useful stepping stone toward the advanced challenge.

- Variational quantum algorithms course

IBM Quantum learning material on the broader family of hybrid variational algorithms.

Hardware execution, transpilation, and error mitigation

- Qiskit documentation

General entry point for circuits, primitives, transpilation, and runtime execution on IBM Quantum.

- Transpile with pass managers

How to transpile circuits for a specific backend and its connectivity.

- Error mitigation and suppression techniques

Overview of the resilience options available through Qiskit Runtime.

- Combine error mitigation options with the Estimator primitive

Tutorial on dynamical decoupling, measurement-error mitigation, gate twirling, and zero-noise extrapolation.

Background reading

- Kandala et al., *Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets*, Nature (2017)

The IBM Quantum paper that inspires this challenge track.

Track 2: Quantum Error Management

Prompts developed and written by Tanner Smith, Collin Frink, Shipra Singh, Eva Gurra

Environmental and intrinsic noise are significant problems for quantum computers that cannot be easily remedied. This kind of noise can result in bit and phase flip errors that make computation difficult. In classical computers, error management is often done through copying the same information onto multiple bits. Such a protocol is not possible in a quantum computer since quantum information cannot be cloned. This is known as the **No Cloning Theorem**.

Nonetheless, error management is very much needed as quantum computers are more error-prone than classical computers. This means that a quantum algorithm running many operations across a large array of qubits is likely to produce incorrect results. Therefore, detecting and suppressing errors, and developing error correction codes for qubit arrays is critical for large-scale operation.

Error suppression reduces the likelihood of hardware errors in the first place, typically using methods to optimize state preparation, control pulses, readout, and layout selection. **Error mitigation** strategies often leverage additional post-processing to suppress the effects of noise. Finally, there is **quantum error correction** (QEC), which is the field that works to build protocols for identifying and/or correcting errors in the quantum computer.

Getting started - Beginners

In this challenge, you will explore one quantum error management strategy: quantum error mitigation (QEM). Unlike QEC, quantum error mitigation strategies introduce minimal quantum resource overhead to reduce noise effects. This facilitates the implementation of different QEM strategies on real-world quantum hardware! In the future, on large-scale quantum computers, QEM will likely be jointly implemented with QEC protocols to optimize quantum resource efficiency.
You will work through three Jupyter notebooks:

1. Start with **Error_Mitigation_Beginner_Primer.ipynb** which is not graded. This notebook starts from the circuits you have seen in the bootcamp and provides a step-by-step walk through for implementing your first QEM strategy!
2. The **SLC_Magnetization_Partner_Lab.ipynb** notebook is graded and will guide you through implementing other QEM strategies on more complex application circuits.
3. Finally, work through the the Noise Busters hackathon challenge notebook.

Your target

Work through the provided QEM Beginner Jupyter Notebooks and complete the listed prompts. You will be scored based on how effective your chosen QEM strategies are in mitigating the provided noisy quantum circuits.

Intermediate - Biased Noise Decoders

Quantum error correction (QEC) codes are constructed in a variety of ways. One such formalism is the stabilizer formalism which involves a set of Pauli operators that leave the valid code space unchanged after measurement. The syndromes are the measurement outcomes obtained by the stabilizers.

In order to analyze the syndromes' noisy measurement data, QEC relies on decoders. Typical decoding algorithms include Minimum Weight Perfect Matching (MWPM) and Union-Find (UF). These algorithms usually assume an unbiased, identically distributed noise model, e.g. bit-flip ($\hat{X}$) and phase-flip ($\hat{Z}$) errors are treated as equally likely. However, real hardware is rarely symmetric and can be designed to have a significant noise bias. Additionally, the noise bias may not even be known.

In this challenge, follow the **Biased_Noise_Decoder_Intermediate_Prompt.ipynb** Jupyter notebook which consists of three parts. The first part walks you through some of the basics of error detection and decoders. The second tasks you with co-designing a QEC code, circuit, and decoder for a system with a known biased noise model. The final section gives you various unknown, biased noise models and tasks you with estimating the noise model and generating a decoder that can handle various biased noise models.

Your target

Work through the provided Intermediate Jupyter Notebook and complete the prompts listed under each of the two targets: known biased noise model and unknown biased noise model.

Advanced - Develop a QEC Visualizer

QEC is essential for building fault-tolerant quantum computers, but it is also one of the most mathematically dense areas of the field. Concepts such as stabilizers, syndromes, error propagation, and decoding are difficult for students and new researchers to grasp from formalisms alone. Good visualization can turn these abstractions into something intuitive, lowering the barrier to entry and helping experienced researchers reason about complex protocols.

Some QEC visualization tools already exist. One example is Craig Gidney's Crumble tool, which lets users lay out the data and syndrome qubits of a stabilizer code (surface code, color code, etc.) and watch how stabilizers, data qubits, and gates interact spatially over a syndrome extraction cycle. Many other areas of QEC would benefit from similar treatment.

Your target

Build an interactive or animated visualization tool that explains or demonstrates a concept in quantum error correction. Your tool must use Qiskit to generate, simulate, or analyze the underlying quantum circuits or error-correction data that drive the visualization.

Possible Directions

You are free to choose any QEC concept. Some ideas to get you started:

- **Error propagation and decoding:** Visualize how errors spread through a surface code (or another code) during syndrome extraction, and show how a decoder such as minimum-weight perfect matching (MWPM) pairs syndrome flips and identifies error chains for a given set of defects.
- **Lattice Surgery:** Given a fixed device layout (surface code patch orientation and physical qubit placement) and an input logical circuit, show step by step how lattice surgery operations are carried out: ancilla patch allocation, merges and splits, measurements, and ancilla discarding. Bonus: illustrate the flow of logical information or phase through the operations. Daniel Litinski's paper "A Game of Surface Codes" may be a helpful starting point to understanding lattice surgery visualization.
- **Your own idea:** Any visualization that offers real educational or demonstrative value to the QEC community is welcome. Examples might include logical operator tracking, code-switching, magic state distillation, or comparing code families.

Track 3: Cryptography

Prompts developed and written by Eva Gurra, Nicholas Papadopoulos, and Matthew Fox

Cryptography is the art of concealing information from any unauthorized third party while exchanging information between two parties. Randomly chosen binary strings or numbers from a large set are known as **keys**, and they provide the security for cryptographic protocols for encryption of information, authentication, and sharing.

Getting started - Beginner

You will explore a protocol first laid out by Charles Bennett and Gilles Brassard in 1984 for sharing states measured in different bases from Alice to Bob. Here, you want to distribute a quantum key between two parties who wish to communicate hence the same "quantum key distribution" (QKD).

The protocol will look something like this: Alice starts by choosing a random sequence of bits and a randomly chosen basis to encode each bit. Bob, who wants to receive the information, also randomly selects a measurement basis. Over a classical channel, Alice can check that Bob received the qubits and compare the bases they used for measuring. Without revealing the bit values, they discard the ones where the choice of basis was different. Depending on the level of security, they can also compare a subset of the bit values to check whether eavesdropping has occurred.

For this challenge, first go through and answer the questions in the provided **quantum-key-distribution_fallfest.ipynb**Jupyter notebook. Then implement a newer variation of the BB84 protocol called SARG04. This protocol uses the same four quantum states as BB84 but instead of Alice and Bob checking whether their bases match, Alice will announce a pair of states - one that she actually sent and one from the opposite basis. Bob will then check his measurement result to rule out one of them. To better visualize Quantum Key Distribution, refer to https://github.com/taoq-ai/qkd-playground. This package can help you understand the protocols and aid with solving the challenge.

Your target

Implement the SARG04 protocol using Qiskit in the presence of eavesdropping and discuss the practical implementation differences between SARG04 and BB84.

Intermediate

Shor's algorithm, developed in 1994 by the American mathematician Peter Shor, completely revolutionized how powerful quantum computing could be and its impact on how we secure our data. In particular, Shor's algorithm would be able to crack two foundational problems in cryptography:

- **Integer factorization problem** (IFP): breaks RSA encryption which relies on finding the prime factors
- **Elliptic curve discrete logarithm problem** (ECDLP): breaks elliptic curve cryptography (ECC) encryption

In this prompt, you will work on using Shor's algorithm to solve the ECDLP. An elliptic curve equation over a finite field $\mathbb{F}_p$ (where p $>$ 2 and prime) is the set of points that satisfies an equation

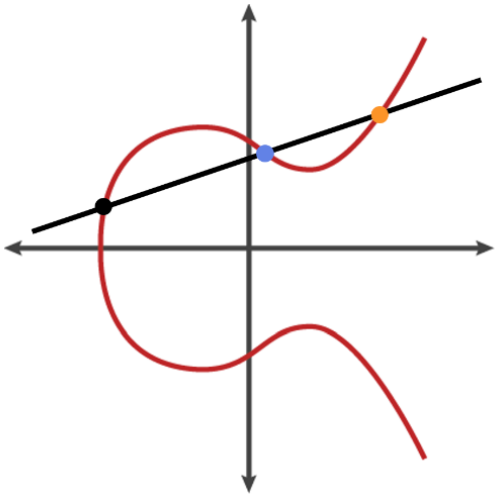


Figure 1: Figure of an elliptic curve showing the property that any non-vertical line will intersect the curve in at most three places. Go here for further visualization of elliptic curves.

like:

$$y^2 = x^3 + ax + b \pmod{p}$$

Two points over an elliptic curve can be added and result in another point on the curve, known as elliptic curve point addition. If we take a point $\mathbf{P}$ and add it to itself k times, we get:

$$\mathbf{Q} = \mathbf{P} + \mathbf{P} + \ldots = k * \mathbf{P}.$$

The key idea here is that multiplying a point by an integer *generates* another point in its subgroup over the elliptic curve. This operation is very fast, but the inverse problem $k = P/Q$ is very hard to solve.

Your target

Reduce the ECDLP problem to a period-finding problem and apply Shor's algorithm in Qiskit for a 7-bit elliptic curve.

Directions

- Use the elliptic curve $y^2 = x^3 + 7 \pmod{p}$ where $p = 127$, which requires exactly 7 bits to represent coordinate elements.
- Select a generator point P and a secret 7-bit private key scalar k and compute the public key point is $Q = kP$.
- To begin cracking the problem $Q = kP$, you must reduce it to a period-finding problem so that you can apply Shor's algorithm. You may refer to **shors-algorithm_fallfest.ipynb** on more background on Shor's algorithm. Unlike RSA factoring, ECDLP requires a two-parameter periodic function.

Advanced - secure communication using quantum-classical protocols

The most commonly used algorithms for key generation, RSA and ECDLP, are *not* resistant to quantum attacks in the presence of a fault-tolerant quantum computer that can implement Shor's algorithm. Nonetheless, institutions that employ these encryption methods cannot simply switch over to a purely quantum infrastructure over night, therefore it is practical to consider both classical post-quantum cryptographic algorithms and hybrid techniques for safe communication.

Your target

In this prompt, simulate the transmission of a message over a classical channel in a way that is robust against quantum attacks using Qiskit.

Possible Directions

These are merely some ideas you can consider, but we encourage you to come up with your own!

- **Hybrid quantum-classical VPN**: securely generate and exchange private keys using Quantum Key Distribution or Quantum Random Number Generators and combine with a post-quantum cryptography (PQC) algorithm. You could consider combining keys generated with a PQC algorithm and QKD using a Key Derivation Function. Then, simulate the transmission of a classical message over an untrusted network with potential eavesdropping using the quantum key.
- **Two-factor authentication**: Simulate an environment where a user (Alice) wants to authenticate her identity to a secure classical server (Bob) in the presence of eavesdropping.